

# Week 2. Kubernetes Networking & CNI Fundamentals

## 1. Kubernetes Networking Model

쿠버네티스 네트워킹 모델은 모든 파드(Pod)가 NAT(네트워크 주소 변환) 없이 고유한 IP를 가지고 서로 통신할 수 있는 플랫(Flat) 네트워크 구조를 제공한다.

### 핵심 기본 원칙

쿠버네티스는 네트워크 구현 방식과 관계없이 다음 원칙을 반드시 보장해야 한다.

- **파드 간 통신 (Pod-to-Pod)**  
: 모든 파드는 NAT를 사용하지 않고도 다른 파드와 직접 통신할 수 있다.(윈도우 호스트 네트워크 파드는 예외)
- **노드-파드 간 통신 (Node-to-Pod)**  
: 노드의 에이전트는 해당 노드 내의 모든 파드와 통신할 수 있다.
- **IP 일관성 (Pod IP Consistency)**  
: 파드가 바라보는 자신의 IP 주소와 다른 파드가 바라보는 그 파드의 IP 주소가 동일하다.

### 주요 구성 요소

#### 파드(Pod)와 네임스페이스

- IP-per-Pod 모델  
: 클러스터 내 모든 파드는 고유한 IP 주소를 할당받는다.
- 네트워크 공유  
: 하나의 파드 안에 속한 여러 컨테이너는 네트워크 네임스페이스를 공유하므로 `localhost` 와 포트를 통해 서로 통신할 수 있다.

#### CNI (Container Network Interface)

- 쿠버네티스 자체는 직접 네트워크를 구축하지 않고, 표준 인터페이스인 CNI 플러그인을 통해 파드에 IP를 부여하고 라우팅을 설정한다.

- 파드위 네트워크 네임스페이스 자체를 만드는 건 CRI(컨테이너 런타임 인터페이스)이고, 그 안에 실제 네트워크(IP 할당, 라우팅, 인터페이스 연결)를 구성하는 건 CNI
- 대표적인 CNI: Calico, Cilium, Flannel ...

## Service, EndpointSlice, kube-proxy

- 파드의 IP는 생성과 소멸에 따라 수시로 변하므로, 안정적인 접근을 위해 Service 오브젝트를 사용해 고유한 가상 IP (ClusterIP)를 제공한다.
- 쿠버네티스는 서비스의 현재 백엔드 파드 목록을 EndpointSlice 오브젝트로 자동 관리한다. 서비스가 어떤 파드로 연결되는지는 이 EndpointSlice를 통해 추적된다.
- `kube-proxy` 는 Service와 EndpointSlice를 모니터링 하다가, 서비스로 들어오는 트래픽을 실제 백엔드 파드로 라우팅(로드밸런싱)하는 역할을 담당한다.
- 일부 파드 네트워크 구현체는 kube-proxy 대신 자체 서비스 프록시를 사용하기도 한다.

## 네트워크 정책 (Network Policy)

- 기본적으로 모든 파드는 서로 통신이 가능하지만, 보안 강화를 위해 특정 파드 간의 트래픽 허용 및 차단 규칙을 OSI 3~4계층 수준에서 제어할 수 있다.
- NetworkPolicy는 일반적으로 CNI 플러그인이 구현한다. 일부 단순한 CNI는 이를 지원하지 않는데, 이 경우 API 자체는 존재하지만 실제로는 아무 효과가 없다.

## 2. kubelet / CRI / CNI 동작 흐름

쿠버네티스에서 Pod가 생성될 때 **kubelet**, **CRI**, **CNI**는 상호작용하며 파드와 네트워크를 구성한다.

```
apiserver → kubelet → CRI(container runtime) → CNI 플러그인 → Pod 네트워크 완성
```

1. kubelet
  - : apiserver를 감시하다가 자신의 노드에 배치된 새 Pod 스펙을 감시
2. kubelet → CRI
  - : CRI에게 sandbox(pause 컨테이너) 생성 요청
3. CRI
  - : pause 컨테이너를 생성·구동하며, 이 과정에서 Pod의 네트워크 네임스페이스가 함께

생성. pause 컨테이너는 이후 이 네임스페이스를 계속 붙잡아 유지하는 anchor 역할을 함.

#### 4. CRI → CNI

: 생성된 네트워크 네임스페이스를 CNI 플러그인에게 넘기며 **ADD** 커맨드 실행

#### 5. CNI 플러그인

: 네임스페이스 안에 veth 생성, IP 할당, 라우팅/브리지 설정

#### 6. CNI → CRI

: 할당된 IP 등 결과를 CRI에 반환

#### 7. CRI → kubelet

: 완성된 sandbox 정보를 반환 → 이후 실제 컨테이너들이 이 sandbox 네트워크 네임스페이스를 공유하며 생성됨

## 3. Pod Network / Pod CIDR / IPAM

### Pod Network

쿠버네티스 클러스터 내부의 모든 Pod가 NAT(네트워크 주소 변환) 없이 서로 직접 통신할 수 있도록 연결된 가상의 단일 L3 네트워크 영역이다.

쿠버네티스가 표준으로 채택하고 있는 네트워크 모델(Kubernetes Network Model)의 핵심 원칙을 만족하는 결과물이다.

- **L3 Direct 통신**: 임의의 Pod A가 다른 Node에 있는 Pod B로 패킷을 보낼 때, 중간에 포트 포워딩이나 주소 변환(NAT)을 거치지 않고 가야 한다.
- **IP 일관성**: Pod 내부에 진입해 **ifconfig** 로 확인한 본인의 IP와, 외부의 다른 Pod가 이 Pod를 목적지로 바라보는 IP가 완전히 일치해야 한다.
- **물리적 구현**: 이 가상 네트워크를 실제로 동기화하고 구성하는 물리적인 주체가 바로 **CNI(Container Network Interface) 플러그인**(Calico, Cilium, Flannel 등)이다.

### Pod CIDR

클러스터 전체 혹은 특정 노드에 할당 가능한 가상 IP 주소의 범위를 의미하며, CIDR 표기법으로 정의된다.

IP 충돌을 방지하기 위해 계층적인 구조로 쪼개어 관리된다.

- **Cluster CIDR**: 클러스터 초기화 단계에서 지정하는 거대한 가상 IP 풀이다.

- **Node CIDR:** `kube-controller-manager` 내의 `node-ipam-controller` 가 새로운 Node가 클러스터에 조인할 때 Cluster CIDR에서 떼어내 할당해 주는 서브넷 블록이다. 이 설정은 각 Node 오브젝트의 `.spec.podCIDR` 필드에 기록된다.

## IPAM (IP Address Management)

개별 Node 내부에서 Pod가 생성되거나 삭제될 때, 부여받은 Node CIDR 범위 내에서 IP를 실제로 할당하고 회수하는 추적/관리 메커니즘이다.

CNI 플러그인의 하부 모듈 형태로 동작하며, 아키텍처에 따라 크게 세 가지 방식으로 분류된다.

- **host-local 방식:** 가장 단순한 레거시 형태로, Node 로컬 디스크 (`/var/lib/cni/networks/`) 내의 파일 시스템을 통해 IP 할당 상태를 기록한다. 정적인 Node CIDR 안에서만 정상 동작한다.
- **CNI 자체 확장 방식 (Calico IPAM 등):** 쿠버네티스 데이터스토어(etcd)나 CRD를 활용한다. Node CIDR 경계를 넘어, 특정 Node에 Pod가 밀집될 경우 다른 Node의 미사용 IP 블록(Block)을 동적으로 빌려오는 등의 유연한 스케일링이 가능하다.
- **Cloud Provider 연동 방식 (AWS VPC CNI 등):** 가상 IP 대신 Cloud Provider의 실제 인프라 IP(예: AWS ENI secondary IP)를 이용한다. 쿠버네티스의 IPAM이 VPC 내부의 IP 관리 시스템과 다이렉트로 연동되어 구동된다.

## 전체적인 동작 흐름

1. `kube-controller-manager` 가 새로운 Node에게 `.spec.podCIDR` 을 지정한다. (**Pod CIDR**)
2. CRI(컨테이너 런타임)의 지시를 받은 CNI 플러그인이 로컬 또는 원격 저장소에서 비어 있는 IP 한 개를 조회 및 선점한다. (**IPAM**)
3. 선점한 IP를 가상 인터페이스(veth)에 바인딩하고 라우팅 테이블을 갱신하여, 해당 Pod가 클러스터 전체와 격리 없이 패킷을 주고받을 수 있도록 인터커넥트한다. (**Pod Network**)

## 4. kube-proxy와 Service

### Service

#### Service가 왜 필요한가?

Pod는 Deployment 등에 의해 동적으로 생성/삭제되며, 각자 고유 IP를 갖지만 그 IP는 언제든지 바뀔 수 있다. 또한 프론트엔드 입장에서는 지금 살아있는 백엔드 Pod의 IP가 무엇인

지를 계속 추적해야 하는 문제가 생긴다.

**Service**는 이 문제를 해결하기 위한 추상화로, selector로 정의된 Pod 집합에 대해 안정적인 접근 방법을 제공한다.

## Service의 동작 방식

1. Service를 생성하면 쿠버네티스가 **ClusterIP**(가상 IP)를 할당한다.
2. Service의 컨트롤러가 selector와 일치하는 Pod를 계속 스캔하면서, 그 결과를 **EndpointSlice** 오브젝트에 갱신한다.
3. 클라이언트는 ClusterIP로 접속하면 되고, 실제 어떤 Pod로 연결될지는 EndpointSlice 기반으로 라우팅된다.

## kube-proxy

### kube-proxy의 역할과 배포 방식

- 모든 노드에서 실행되는 **DaemonSet**으로 배포된다. 클러스터의 모든 노드가 Service 기반 통신을 처리해야 하므로, 노드마다 하나씩 있어야 하는 성격 때문이다.
- API 서버에서 Service와 EndpointSlice(구 Endpoints)의 변경을 계속 watch하다가, 변경이 생기면 자신이 실행 중인 노드의 네트워킹 규칙(iptables 등)을 갱신한다.
- 결국 백엔드 Pod로 트래픽을 전달(라우팅/로드밸런싱)하는 것이 최종 역할이다.

### kube-proxy의 동작 모드

| 모드                    | 특징                                                                                                                                                                                                                                            |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>userspace</b>      | 요청이 오면 iptables를 거쳐 kube-proxy가 <b>직접 프록시 서버 프로세스를 띄워</b> 로드밸런싱을 수행하는 방식. 가장 오래된 레거시 모드이며 현재는 권장되지 않는다.                                                                                                                                       |
| <b>iptables (기본값)</b> | 리눅스 iptables로 서비스로 들어오고 나가는 패킷을 제어하는 방식. kube-proxy는 API 서버를 통해 얻은 Service/Pod 정보를 바탕으로 <b>iptables 규칙만 업데이트</b> 하고, 실제 트래픽 처리는 커널 단에서 이루어지므로 userspace 모드보다 안정적이다. 기본 로드밸런싱 방식은 랜덤이며, Service 수가 많아지면 규칙 수가 선형( $O(n)$ )으로 늘어나 성능이 저하될 수 있다. |
| <b>IPVS</b>           | 리눅스 IPVS 기능을 사용하는 방식. 클라이언트 요청을 가상 서버 IP로 전달하면 IPVS가 로드밸런싱 알고리즘에 따라 올바른 Pod로 전달한다. 해시 테이블을 기본 자료구조로 사용하고 커널 스페이스에서 동작하기 때문에 iptables 대비 성능·확장성이 좋다( $O(1)$ 에 가까움). 단, 노드의 커널 모듈에서 IPVS가 활성화되어 있어야 사용 가능하다.                                    |

## 왜 Service만으로는 통신이 안 되는가

- ClusterIP는 가상 IP이기 때문에, Pod의 veth나 노드의 eth0는 이 주소로 어떻게 라우팅해야 할지 알 수 없다.
- 라우팅 테이블에 없는 목적지는 상위 네트워크(gateway)로 전달되지만, gateway 역시 ClusterIP를 모르기 때문에 트래픽은 결국 전달되지 못한다.
- 따라서 "이 ClusterIP로 온 트래픽을 실제 어떤 Pod로 보내야 하는지"를 각 노드에게 알려주는 주체가 필요하며, 그 역할을 kube-proxy가 수행한다.

## 5. Service → Pod Packet Flow 추적

```
[Client Pod]
| (Dst: Service IP)
▼
[Kernel / Netfilter (iptables / IPVS)] → DNAT 실행 (Dst:
Service IP → Target Pod IP)
|
▼
[CNI Network (Bridge / Overlay)] → 라우팅 및 캡슐화 (VXLAN
등)
|
▼
[Target Pod] → 응답 시 conntrack에 의해 역변환 (Src: Target
Pod IP → Service IP)
```

### 1. 패킷 생성 (Client Pod)

- Client Pod에서 요청을 보낸다.
- **Source IP:** Client Pod IP / **Destination IP:** Service ClusterIP
- 패킷은 Pod 내부의 `eth0` veth 인터페이스를 지나 노드의 Root Network Namespace로 빠져나온다.

### 2. 트래픽 가로채기 및 DNAT (Node Kernel)

- 패킷이 노드의 커널(Netfilter)을 통과할 때, `kube-proxy`가 사전에 정의해둔 규칙 (iptables 또는 IPVS)을 만난다.
- DNAT (Destination Network Address Translation)가 발생한다.

- `Destination: Service IP:Port` → `Destination: 백엔드 Pod IP:Port`
- 커널의 `conntrack` (**Connection Tracking**) 테이블에 이 연결 상태(요청 IP, 변환된 Pod IP 등)가 기록된다.

### 3. 노드 간 라우팅 및 패킷 전달 (CNI)

- 목적지 Pod IP가 결정되면 CNI(Calico, Cilium, Flannel 등)가 패킷 경로를 제어한다.
- **동일 노드 내 Pod**: 노드의 `bridge` 나 `veth` 가상 인터페이스를 통해 바로 대상 Pod로 전달된다.
- **타 노드 Pod**: CNI 방식에 따라 라우팅 방식이 달라진다.
  - **Overlay (VXLAN 등)**: 원래 패킷을 노드 간 IP 패킷으로 캡슐화하여 타 노드로 전송 후 복호화.
  - **Non-Overlay (BGP 등)**: 별도 캡슐화 없이 BGP 테이블을 타고 목적지 노드의 eth0로 직접 전달.

### 4. 목적지 도착 및 응답 패킷의 역변환 (Return Path)

- 패킷이 Target Pod의 Network Namespace로 들어가 애플리케이션에 전달된다.
- Target Pod가 응답 패킷을 생성한다.
  - **Source IP**: Target Pod IP / **Destination IP**: Client Pod IP
- 돌아오는 길에 노드 커널의 `conntrack` 테이블을 조회하여 **Reverse DNAT**를 수행한다.
  - `Source: Target Pod IP` → `Source: Service IP` 로 변경
- Client Pod는 자신이 요청했던 Service IP로부터 응답을 받은 것으로 인식하게 된다.

#### ▼ 참고자료

<https://kubernetes.io/ko/docs/concepts/services-networking/>

<https://kubernetes.io/ko/docs/concepts/cluster-administration/networking/>

<https://kubernetes.io/ko/docs/concepts/services-networking/service/>

## 질문

1. 파드가 재시작되면 IP가 바뀐다고 했는데, 같은 파드인데 IP만 바뀌는 건지 아니면 완전히 새 파드로 취급되는 건지?
2. iptables 모드가 기본값이라고 했는데, 실무에서는 IPVS를 더 많이 쓰는 편인지?